

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Захматов Юрий

К3341

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд (можно взять и [мой](#))
2. С учётом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
 - модели
 - представления
 - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход выполнения работы

1. Анализ предметной области и проектирование структуры проекта

На первом этапе была проанализирована предметная область — сервис аренды недвижимости с возможностью бронирования, отзывами, чатами между пользователями и платёжными операциями. Выделены основные сущности: пользователь, объект недвижимости, бронирование, отзыв, платёж, чат, сообщение, удобства, изображения. Разработана ER-модель, определены связи (@OneToMany, @ManyToOne, @ManyToMany).

Проект построен по многослойной архитектуре:

контроллеры (controllers) — обработка HTTP-запросов;

сервисы (service) — бизнес-логика;

репозитории (repositories) — работа с БД через Spring Data JPA;

DTO (dto) — объекты передачи данных;

сущности (entities) — маппинг на таблицы БД.

2. Настройка окружения и конфигурация проекта

Создан Spring Boot проект с использованием Gradle. Подключены зависимости: Spring Web, Spring Security, Spring Data JPA, PostgreSQL Driver, Lombok, Validation, JWT для JWT, Flyway для миграций. Настроен файл application.yml (параметры подключения к БД, настройки JWT: access.secret, refresh.secret, времена жизни токенов, логирование). Настроена JPA (ddl-auto: validate).

3. Реализация миграций базы данных (Flyway)

Созданы SQL-миграции для инициализации схемы БД: таблицы _user_ (с полями username, email, passwordHash, роли, isRenter/isLandlord), property, booking, review, payment, chat, message, amenity, property_amenity_link, refresh_token. Добавлены индексы, внешние ключи, последовательности. Настроено автоматическое применение миграций при старте приложения.

4. Реализация сущностей (Entity) и репозитория (JpaRepository)

Созданы JPA-сущности:

UserEntity с реализацией UserDetails для Spring Security.

PropertyEntity (привязка к владельцу, список удобств, изображений, отзывов).

BookingEntity (связь с арендатором, объектом, оплатой, статусы PENDING/CONFIRMED/CANCELLED/COMPLETED/REJECTED).

ReviewEntity (рейтинг, комментарий, связь пользователя и объекта).

PaymentEntity (сумма, статус PENDING/APPROVED/REJECTED).

ChatEntity и MessageEntity (чат между двумя пользователями, сообщения с временными метками).

AmenityEntity, PropertyImageEntity, RefreshTokenEntity.

Для каждой сущности написан репозиторий (интерфейс, расширяющий JpaRepository). В репозиториях добавлены кастомные методы:

UserRepository — поиск по username, проверка существования email.

PropertyRepository — фильтрация по множеству параметров (цена, площадь, тип, локация) и поиск по ключевому слову через @Query.

ChatRepository — поиск чата между двумя пользователями, все чаты пользователя с пагинацией.

MessageRepository — получение сообщений с сортировкой по времени.

BookingRepository — поиск бронирований по объекту и по арендатору.

5. Реализация DTO и маппинга

Созданы record-классы DTO для каждого основного сценария:

Запросы: UserUpdateRequest, CreatePropertyRequest, UpdatePropertyRequest, BookingCreateRequest, ReviewCreateRequest, SendMessageRequest, ChatCreateRequest.

Ответы: UserResponse, UserShortResponse, PropertyResponse, BookingResponse, ReviewResponse, PaymentResponse, MessageResponse, ChatResponse, ImageResponse, AuthResponse, RefreshTokenResponse.

Реализовано ручное маппинг сущности → DTO в сервисах с помощью методов mapToResponse(...). Для PropertyResponse дополнительно подгружаются изображения и названия удобств.

6. Реализация аутентификации и авторизации (JWT + Spring Security)

6.1. Настроен SecurityConfiguration:

отключён CSRF;

установлена stateless сессия;

открыты эндпоинты: /api/v*/auth/login, /api/v*/auth/register, /api/v*/auth/refresh, GET /api/v*/properties/**;

эндпоинты /api/v*/admin/** — только для роли ADMIN;

остальные — требуют аутентификации.

6.2. Реализован JwtAuthenticationFilter (наследник OncePerRequestFilter):

извлечение JWT из заголовка Authorization: Bearer <token>;

валидация access токена;

установка UsernamePasswordAuthenticationToken в контекст Security.

6.3. Создан CustomUserDetailsService, загружающий UserEntity из БД по username.

6.4. Реализован JwtService:

генерация access и refresh токенов с разными секретами и временем жизни);

извлечение claims (username, userId, role);

валидация токенов.

6.5. Реализован RefreshTokenService и сущность RefreshTokenEntity:

хранение refresh токенов в БД;

поддержка механизма revoke (при использовании старого токена он аннулируется);

автоматическое удаление предыдущих токенов пользователя при создании нового.

6.6. Написан AuthService и AuthController с эндпоинтами:

POST /api/v1/auth/register — регистрация нового пользователя (пароль хэшируется BCryptPasswordEncoder), возвращается пара токенов.

POST /api/v1/auth/login — аутентификация, возврат токенов.

POST /api/v1/auth/refresh — обновление access токена по refresh токену с выдачей новой пары.

POST /api/v1/auth/logout — аннулирование refresh токена.

7. Реализация CRUD-операций и бизнес-логики

7.1. Пользователи (UserService, UserController)

GET /api/v1/users/me — получение текущего пользователя.

PATCH /api/v1/users/me — частичное обновление (firstName, lastName, email, phoneNumber) с проверкой уникальности email.

GET /api/v1/users/me/bookings — список бронирований текущего пользователя (пагинация).

GET /api/v1/users/me/properties — список объявлений текущего пользователя.

7.2. Управление недвижимостью (PropertyService, PropertyController)

GET /api/v1/properties — список всех доступных объектов (пагинация).

GET /api/v1/properties/{id} — детальная информация об объекте (с подгрузкой владельца, изображений, удобств, среднего рейтинга).

POST /api/v1/properties — создание объявления (только для авторизованных). Реализована логика: сохранение объекта, привязка удобств (AmenityService.findOrCreateAmenities), сохранение изображений.

PUT /api/v1/properties/{id} — полное обновление объявления (с проверкой прав: только владелец или ADMIN).

DELETE /api/v1/properties/{id} — удаление объявления (каскадное удаление изображений, бронирований, отзывов через orphanRemoval).

PUT /api/v1/properties/{id}/images — массовое обновление изображений (замена всего списка).

DELETE /api/v1/properties/{propertyId}/images/{imageId} — удаление конкретного изображения.

Фильтрация и поиск:

GET /api/v1/properties/filter — динамические фильтры: available, type, country, region, city, nearestSubway, minPrice, maxPrice, minSquare, maxSquare, а также поиск по ключевому слову в title/description. Реализовано через JPQL @Query с параметрами.

7.3. Бронирования (BookingService, BookingController)

POST /api/v1/properties/{propertyId}/bookings — создание бронирования (только для арендатора). Статус по умолчанию — PENDING.

GET .../bookings — список всех бронирований объекта (пагинация).

GET .../bookings/{id} — получение конкретного бронирования с проверкой прав (арендатор или ADMIN).

PATCH .../bookings/{id}/cancel — отмена бронирования (только арендатор или ADMIN).

PATCH .../bookings/{id}/confirm — подтверждение бронирования (только владелец объекта или ADMIN). При подтверждении автоматически создаётся платёж через PaymentService.createPayment().

PATCH .../bookings/{id}/reject — отклонение бронирования (владелец или ADMIN).

PATCH .../bookings/{id}/complete — завершение (владелец или ADMIN), доступно только после даты окончания.

7.4. Отзывы (ReviewService, ReviewsController)

POST /api/v1/properties/{propertyId}/reviews — создание отзыва (только авторизованные). При создании автоматически пересчитывается средний рейтинг объекта: увеличивается totalReviews, ratingSum, новое avgRating округляется до двух знаков.

GET /api/v1/properties/{propertyId}/reviews — список отзывов с пагинацией и сортировкой по дате.

DELETE .../reviews/{id} — удаление отзыва (только автор или ADMIN). В текущей версии пересчёт среднего рейтинга при удалении не выполнен — отмечен как TODO.

7.5. Платежи (PaymentService, PaymentsController)

GET /api/v1/properties/{propertyId}/bookings/{bookingId}/payments/{id} — получение информации о платеже (только арендатор или ADMIN).

POST .../payments/{id}/pay — имитация оплаты (заглушка): статус меняется на APPROVED, записывается payedAt. Реальная платёжная система не интегрирована.

7.6. Чаты и сообщения (ChatService, MessageService, ChatController)

POST /api/v1/chats/new — создание чата между двумя пользователями (если чат уже существует — возвращается существующий).

GET /api/v1/chats — список чатов текущего пользователя с пагинацией и последним сообщением.

GET /api/v1/chats/{chatId} — история сообщений (пагинация, сортировка от новых к старым) с проверкой прав (участник или ADMIN).

POST /api/v1/chats/{chatId} — отправка сообщения в чат.

8. Реализация глобальной обработки исключений (GlobalExceptionHandler)

Создан GlobalExceptionHandler с @RestControllerAdvice:

SecurityException → 403 Forbidden;

MethodArgumentNotValidException → 400 Bad Request (ошибки валидации)

EntityNotFoundException → 404 Not Found;

общий Exception → 500 Internal Server Error.

Все ответы обернуты в ErrorResponse(message).

9. Интеграция и тестирование

9.1. Выполнена стартовая миграция Flyway — таблицы созданы.

9.2. Проверена регистрация пользователя (пароль хэшируется BCrypt).

9.3. Проверена аутентификация: после логина возвращаются access и refresh токены.

9.4. Проверено создание объявления недвижимости с удобствами и изображениями.

9.5. Проверены фильтрация и поиск объектов.

9.6. Проверено создание бронирования, подтверждение → автоматическое создание платежа.

9.7. Проверено добавление отзыва → автоматический пересчёт рейтинга.

9.8. Проверен обмен сообщениями между пользователями через чаты.

Вывод по работе

В ходе выполнения работы была спроектирована и реализована бэкенд-часть сервиса аренды недвижимости с полноценной JWT-аутентификацией, ролевой моделью (USER/ADMIN), пагинацией, валидацией входных данных и многослойной архитектурой. Реализованы следующие модули:

- регистрация и аутентификация с refresh-токенами;
- управление профилем пользователя;
- CRUD для объектов недвижимости с фильтрацией и поиском;
- система бронирования с жизненным циклом статусов;
- система отзывов с автоматическим подсчётом рейтинга;
- система чатов и сообщений;
- платёжная заглушка.

Все контроллеры, сервисы и репозитории покрывают заявленный функционал. Код написан с использованием актуальных практик Spring Boot 3+, Spring Security 6, JWT, JPA/Hibernate, Flyway.